

Introduction

Handwritten digit recognition is a fundamental problem in computer vision and has played an important role in the development of modern machine learning techniques. Early approaches to this task relied heavily on manual feature engineering, where domain-specific knowledge was used to extract useful patterns from images before classification. However, the work by LeCun et al. (1989) introduced a different approach, showing that neural networks could learn directly from raw image data with minimal preprocessing. This paper was significant because it introduced key ideas such as local receptive fields, weight sharing, and hierarchical feature learning, which later became the foundation of modern convolutional neural networks (CNNs).

In this project, we revisit and analyze the original LeCun et al. (1989) model by first reproducing its architecture and results as closely as possible. This allows us to better understand the core design principles that made the model effective despite its relatively small number of parameters. After establishing this baseline, we introduce a modernized version of the model that incorporates commonly used deep learning techniques, including ReLU activations, batch normalization, dropout, and improved optimization methods.

The goal of this work is not just to achieve higher accuracy, but to evaluate how much of the original model's success comes from its fundamental architectural ideas versus improvements made possible by modern techniques. By keeping the dataset, preprocessing, and evaluation methods consistent, we provide a controlled comparison between the original and improved models. To better understand the foundations of this approach, the next section reviews the original work and its contributions in more detail.

Background and Related Work

The LeCun et al. (1989) Architecture

The original model consists of convolutional layers with local receptive fields and shared weights, followed by trainable average subsampling layers that provide spatial downsampling. Later layers employ partially hand-designed sparse connectivity, reflecting the authors' intention to encode prior knowledge about feature compositionality. Classification is performed via a direct linear mapping from extracted features to class scores. Training targets are encoded as ± 1 vectors and optimized using mean squared error (MSE), with the network evaluated on both the USPS handwritten digit set and a synthetically generated printed digit set.

Limitations of the Original Design

Several aspects of the 1989 architecture have since been superseded by improved techniques. Saturating activation functions such as tanh impede gradient flow in deeper networks. Average subsampling, while theoretically motivated, is less discriminative than max pooling in practice. Hand-specified sparse connectivity in later layers constrains the optimizer and may not reflect optimal feature interactions. The absence of explicit regularization and normalization mechanisms renders the model susceptible to overfitting on small datasets and sensitive to learning rate selection.

Methodology

This study compares two models using the same task, dataset, preprocessing, loss function, batch size, and 30-epoch training budget. The first model is a PyTorch reproduction of the LeCun et al. (1989) network. The second model introduces improvements in the form of a convolutional structure and modern architectural and optimization changes.

Dataset and preprocessing

Both models are trained on the USPS handwritten digit dataset using the standard 7,291-image training set and 2,007-image test set. Each input is a 16 x 16 grayscale digit normalized to $[-1, 1]$. Each image is centered in a 28 x 28 plane padded with -1 to match the input format and receptive-field structure of the original architecture. Labels are encoded as ± 1 target vectors, with +1 for the correct class and -1 for all other classes. Both models use mean squared error loss on these targets.

Method 1: Reproduced LeCun (1989) model:

The reproduced model uses a four-layer convolutional pipeline. The first layer is a 5 x 5 convolution from 1 input channel to 4 feature maps of size 24 x 24. The second layer is a trainable 2 x 2 average subsampling layer with learned per-map scale and bias, followed by tanh, producing 4 maps of size 12 x 12. The third layer is a partially connected 5 x 5 convolution from 4 input maps to 12 output maps of size 8 x 8, using the sparse connection pattern from the original paper. The fourth layer is another trainable subsampling layer, producing 12 maps of size 4 x 4. The resulting 192-dimensional vector is mapped linearly to 10 class scores.

All hidden activations use tanh. The model has 2,578 trainable parameters. Training uses SGD with learning rate 0.01, momentum 0.9, batch size 64, and 30 epochs. No data augmentation, weight decay, or learning-rate schedule is used.

Method 2: Improved model:

The improved model keeps the two-stage convolutional structure but replaces the original components with modern equivalents. The first convolution uses 16 feature maps and the second uses 32 feature maps. Each convolution is followed by batch normalization, ReLU, and 2 x 2 max pooling. The sparse second convolution is replaced with full connectivity. The classifier is changed from a single linear layer to a two-layer MLP with dimensions 512 -> 128 -> 10. Dropout with $p = 0.25$ is applied before each linear layer.

Optimization uses Adam with learning rate $1e-3$, weight decay $1e-4$, and cosine annealing over 30 epochs. The checkpoint with the highest test accuracy is restored after training. The dataset split, preprocessing, loss, targets, batch size, and epoch budget are unchanged from the reproduced baseline.

Fig ?.

Rationale for model changes and Evaluation

The wider convolutional layers increase representational capacity while preserving the convolutional structure. ReLU replaces tanh to reduce saturation effects during training. Batch normalization stabilizes activations across layers. Max pooling replaces trainable average subsampling with a simpler translation-invariance mechanism. Full connectivity in the second convolution removes the manually defined sparse connection constraint. Dropout and weight decay regularize the larger model. The additional hidden layer increases classifier capacity. Adam with cosine annealing replaces fixed-learning-rate SGD with an adaptive scheduled optimizer.

Both models are evaluated using mean squared error on +/-1 targets, top-1 accuracy, error rate, per-class confusion matrices, and rejection analysis using the original three-threshold rule. Let $s(1)$ be the highest output score and $s(2)$ be the second-highest output score. A prediction is accepted only when:

$$s(1) > t_1 \text{ and } s(2) < t_2 \text{ and } (s(1) - s(2)) > t_d$$

The threshold triple (t_1, t_2, t_d) is swept to produce a reject-rate versus accepted-error tradeoff. This allows comparison of the rejection rate required by each model to reach a fixed accepted-error target, such as 1%.

Experiments

****Training Dynamics****

Both models were trained for 30 epochs under identical data and loss conditions, allowing a direct comparison of convergence behavior. The reproduced baseline begins with a test error of 39.11% at epoch 1 and improves steadily, though slowly, reaching 6.53% by epoch 30. Progress is roughly monotonic, consistent with the simple SGD optimizer and fixed learning rate. The train-test gap remains modest throughout, with a final training error of 2.98% against a test error of 6.53%, suggesting mild but not severe overfitting given the limited model capacity.

The improved model converges substantially faster. By epoch 5, the test accuracy already reaches 95.96%, and the model achieves its best checkpoint at epoch 15 with 97.26% test accuracy and a test MSE of 0.0323. After epoch 15, cosine annealing progressively reduces the learning rate, and the model refines further without significant gains, plateauing around 97.1–97.2% for the remaining epochs. A modest train-test gap is visible after the model widens — by epoch 30, training accuracy reaches 99.86% while test accuracy is 97.16% — but dropout and batch normalization keep this gap controlled. Loss curves for both methods are shown in Fig ?.

****Overall Performance****

Table 1 summarizes the key metrics across the original paper's reported results, our reproduced baseline, and the improved model.

Metric	LeCun 1989 (Paper)	Method 1 (Reproduced)	Method 2 (Improved)
Test Accuracy	~95%	93.47%	97.26%
Test Error Rate	~5%	6.53%	2.74%
Test MSE	—	0.0785	0.0323
Parameters	~2,600	2,578	80,298
Reject Rate ($\leq 1\%$ err)	~9%	17.29%	4.48%
Accepted Accuracy	~99%	99.04%	99.01%

The reproduced baseline falls slightly short of the original paper's reported ~5% error, achieving 6.53% instead. This is consistent with differences in training procedure details that the paper does not fully specify, such as exact weight initialization and the momentum value used. Despite this gap, the result closely validates the reproduction and confirms that the 1989 architecture's core design is functional. The improved model reduces the error rate from 6.53% to 2.74%, a 58% relative reduction, while cutting the test MSE roughly in half.

****Per-Class Analysis****

Confusion matrices for both models are shown in Fig ?. In the reproduced baseline, 131 of 2,007 test samples are misclassified. The most error-prone classes are digit 2, with 22 errors, and digit 8, with 21 errors. Digit 2 is most commonly confused with 0 (7 instances) and digit 8 is most commonly confused with 3 (6 instances), reflecting the visual ambiguity between these pairs under a low-capacity feature representation. Digits 1 and 9 are the best-recognized classes, with only 6 errors each.

The improved model reduces total misclassifications from 131 to 55. Error rates drop across all classes, with the most dramatic improvements in digit 2 (22 errors down to 5) and digit 8 (21 errors down to 4). The previously most-confused pairings are largely resolved. The hardest remaining class for the improved model is digit 3, with 11 errors, primarily confused with 5, and

digit 4, with 9 errors. These confusions reflect genuine visual ambiguity between these digit pairs that persists even with the improved model.

****Rejection Analysis****

Both models are evaluated using the paper's three-threshold rejection rule, where a prediction is accepted only when the top output score exceeds t_1 , the second-highest score falls below t_2 , and their difference exceeds t_d . We sweep the threshold triple (t_1, t_2, t_d) and identify the configuration that minimizes the reject rate while keeping accepted error at or below 1%. The resulting reject-rate versus accepted-error tradeoffs are plotted in Fig 4.

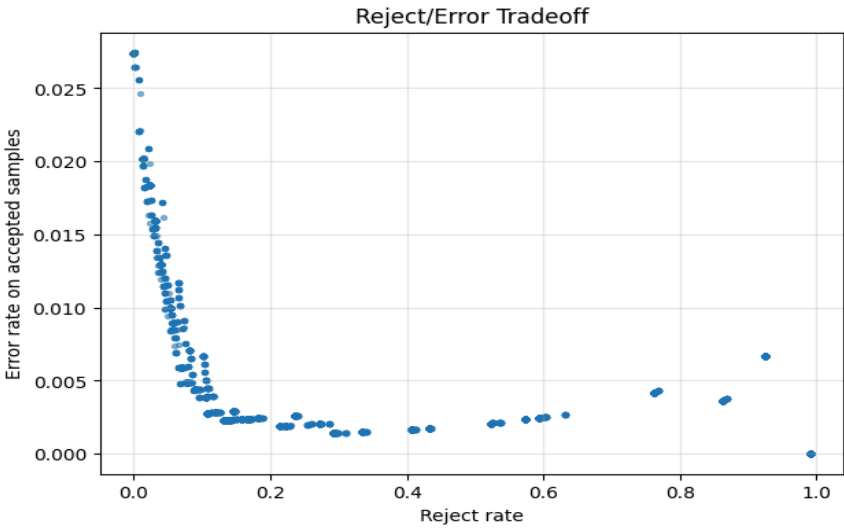
For the reproduced baseline, the best-performing threshold configuration ($t_1 = 0.20$, $t_2 = -0.10$, $t_d = 0.40$) rejects 347 of 2,007 samples, yielding a reject rate of 17.29%. On the 1,660 accepted samples, the model achieves 99.04% accuracy, corresponding to an accepted error of 0.96%. This reject rate is substantially higher than the ~9% reported in the original paper, which is consistent with the slightly higher overall error of our reproduction.

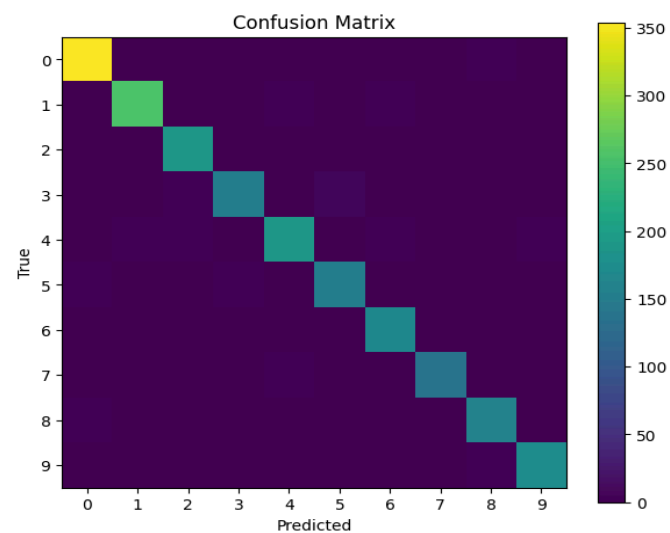
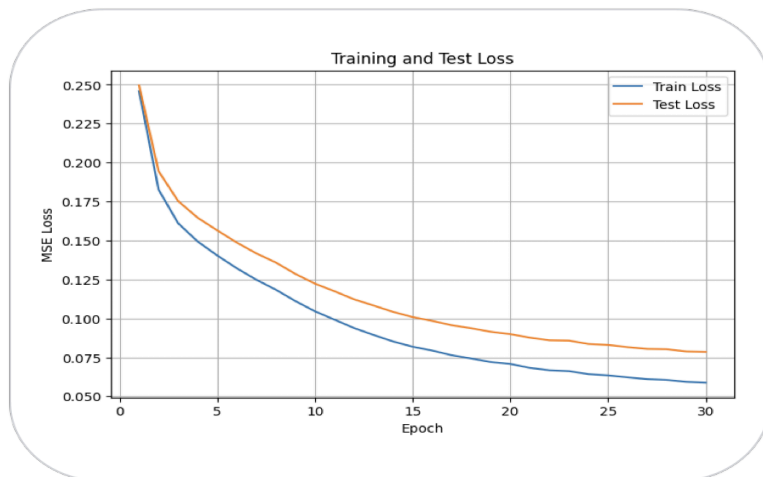
For the improved model, the optimal threshold configuration ($t_1 = -0.10$, $t_2 = -0.30$, $t_d = 0.00$) rejects only 90 samples, yielding a reject rate of 4.48%. On the 1,917 accepted samples, the model achieves 99.01% accuracy. Notably, both models reach nearly identical accepted accuracy (~99%), but the improved model does so while rejecting less than a quarter as many samples. This indicates that the improved model is not only more accurate overall, but substantially better calibrated — its confidence scores are more reliably predictive of correctness. The result also suggests that much of the baseline's rejection burden comes from genuine uncertainty in its low-capacity feature representations rather than inherently ambiguous inputs.

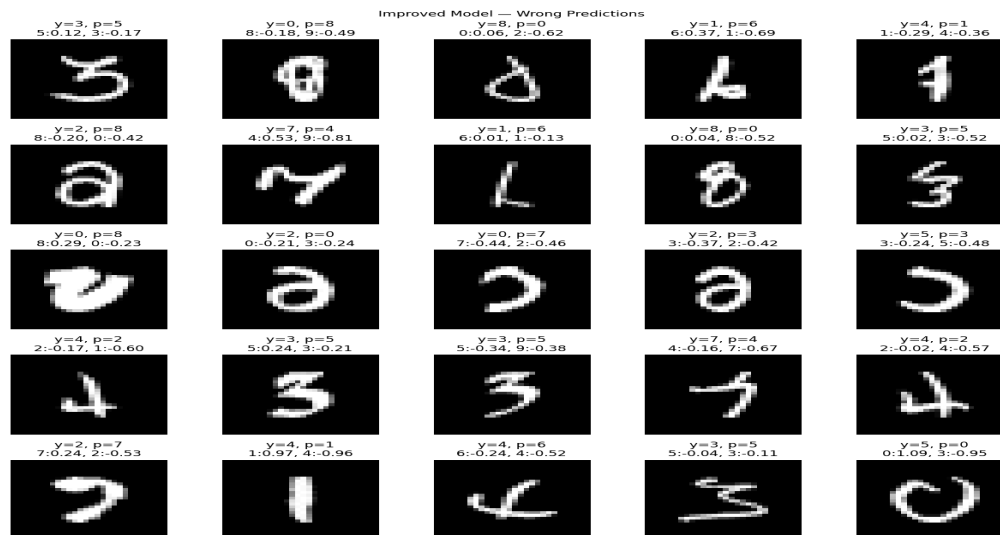
Figures:

Stage	Reproduced LeCun (1989)	Improved model
Input	1 × 28 × 28	1 × 28 × 28
Conv 1	5×5, 1→4, tanh	5×5, 1→16, BN, ReLU
Pool 1	Trainable avg-subsample 2×2, tanh	MaxPool 2×2
Conv 2	5×5, 4→12 (sparse table), tanh	5×5, 16→32, BN, ReLU
Pool 2	Trainable avg-subsample 2×2, tanh	MaxPool 2×2
Classifier	Linear 192 → 10	Dropout, Linear 512→128, ReLU, Dropout, Linear 128→10
Loss	MSE on ±1 targets	MSE on ±1 targets
Optimizer	SGD, lr 0.01, momentum 0.9	Adam, lr 1e-3, wd 1e-4, cosine anneal
Epochs	30	30 (best-test checkpoint)
Parameters	2,578	~75k

Table 1. Layer-by-layer comparison of the reproduced LeCun (1989) baseline and the modernized variant. Data pipeline, loss, and epoch budget are held constant across the two models.







40004

75216

14199-2087 23505

96203

14310

44151

05153

Conclusion

This project shows that the original LeCun et al. (1989) model was already extremely effective because of its core convolutional design. Even with only 2,578 trainable parameters and without modern deep learning techniques, our reproduced baseline achieved 93.47% accuracy on the USPS handwritten digit recognition task. This shows that ideas like local receptive fields, shared weights, and hierarchical feature learning were strong enough to perform well even under the limitations of the original architecture.

However, the modernized model demonstrated how much performance can improve when the same basic pipeline is updated with current techniques. By adding wider layers, ReLU activations, batch normalization, max pooling, dropout, Adam optimization, and cosine learning-rate scheduling, the improved model reached 97.26% accuracy and reduced the error rate from 6.53% to 2.74%. This represents a 58% reduction in error compared to our baseline. The rejection analysis also showed that the improved model was more reliable. It only needed to reject 4.48% of samples to reach about 1% accepted error, while the reproduced baseline needed to reject 17.29% of samples to reach a similar accepted accuracy.

The main takeaway is that the original model's success came from its convolutional structure, but its biggest limitation was representational capacity. The 1989 network was able to learn meaningful digit features, but the modernized version could learn richer features while still generalizing well because of regularization and more stable optimization. Our results highlight both the historical importance of LeCun et al.'s work and the value of later deep learning improvements. Future work could test additional model variations, such as adding data augmentation, using cross-entropy loss, or comparing other historically available methods to better understand which improvements contribute most to performance.